

Kernel Principal Component Analysis

Issues with PCA ?

1. Time Complexity Issue

Finding the eigen value and eigen vector is computationally heavy. we know $C \in \mathbb{R}^{d \times d}$
 $\therefore$ The time complexity of the algorithm grows at a rate of $O(d^3)$ when $d \gg n$

2. Non Linearity Issue

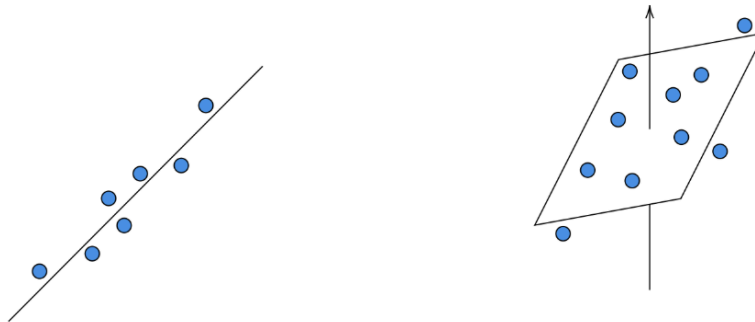


Figure 1: representation of datapoints in subspace

Data may not be necessarily live in a low-dimensional linear subspace, PCA assumes data lies in linear subspace. PCA works well in linear subspace. If we run PCA on non-linear dataset, the error will be very high.

But, Interestingly the solution for both the issue is same!

Addressing the Time Complexity Issue

Datamatrix

$$X = \begin{bmatrix} | & | & & | \\ x_1 & x_2 & \cdot & x_n \\ | & | & & | \end{bmatrix} X \in \mathbb{R}^{d \times n}$$

Calculating Covariance Matrix

$$C = \frac{1}{n} \sum_{i=1}^n x_i x_i^T$$
$$XX^T = \begin{bmatrix} | & | & & | \\ x_1 & x_2 & \cdot & x_n \\ | & | & & | \end{bmatrix} \begin{bmatrix} - & x_1^T & - \\ - & x_2^T & - \\ & \cdot & \\ & \cdot & \\ - & x_n^T & - \end{bmatrix}$$

$$C = \frac{1}{n} XX^T ; C \in \mathbb{R}^{d \times d}$$

$\therefore$ The time complexity of the algorithm in this case is $O(d^3)$

Calculating Gram Matrix

$$k = \begin{bmatrix} - & x_1^T & - \\ - & x_2^T & - \\ & \cdot & \\ & \cdot & \\ - & x_n^T & - \end{bmatrix} \begin{bmatrix} | & | & & | \\ x_1 & x_2 & \cdot & \cdot & x_n \\ | & | & & | \end{bmatrix} = \sum_{i=1}^n x_i^T x_i$$

$$k = X^T X ; K \in \mathbb{R}^{n \times n}$$

$\therefore$ The time complexity of the algorithm in this case is $O(n^3)$

Let, w_k be the eigen vector corresponding to the k^{th} largest eigenvalue(λ_k) of C

$$C \cdot w_k = \lambda_k \cdot w_k$$

$$\left(\frac{1}{n} \sum_{i=1}^n x_i x_i^T \right) \cdot w_k = \lambda_k \cdot w_k$$

$$w_k = \left(\sum_{i=1}^n \frac{x_i^T w_k}{n \cdot \lambda_k} \right) \cdot x_i$$

w_k is a linear combination of datapoints.

$$w_k = X \alpha_k$$

$$\begin{bmatrix} | & | & & | \\ x_1 & x_2 & \cdot & x_n \\ | & | & & | \end{bmatrix} \begin{bmatrix} - & \alpha_{k_1} & - \\ \cdot & & \\ \cdot & & \\ - & \alpha_{k_n} & - \end{bmatrix} = \sum_{i=1}^n \alpha_{k_i} \cdot x_i$$

Observe to get w_k , we need α_k , that indeed need w_k !

How to get α_k without being dependent upon w_k

$$\begin{aligned}
 w_k &= X \alpha_k \\
 C w_k &= \lambda_k w_k \\
 \left(\frac{1}{n} X X^T \right) (X \alpha_k) &= \lambda_k (X \alpha_k) \\
 (X X^T) (X \alpha_k) &= n \lambda_k (X \alpha_k) \\
 X^T ((X X^T) (X \alpha_k)) &= X^T (n \lambda_k (X \alpha_k)) \\
 (X^T X) (X^T X) \alpha_k &= n \lambda_k (X^T X) \alpha_k \\
 k^2 \alpha_k &= n \lambda_k k \alpha_k \\
 k \alpha_k &= n \lambda_k \alpha_k
 \end{aligned}$$

if we can find α_k that satisfies

$$k \alpha_k = n \lambda_k \alpha_k$$

we know length of w_k is 1. what is length of α_k ?

$$\begin{aligned}
 w_k &= X \alpha_k \\
 w_k^T w_k &= (X \alpha_k)^T (X \alpha_k) \\
 w_k^T w_k &= \alpha_k^T (X^T X) \alpha_k \\
 1 &= \alpha_k^T k \alpha_k
 \end{aligned}$$

We wanted the Eigenvectors of XX^T but now we have solved the Eigen equation of K , where K is just $X^T X$. Now, somehow this also involves λk . We need to know λk for which is the Eigen value of XX^T . But we are only solving the Eigen equation for XX^T . So is there any relation between eigen values of XX^T and $X^T X$.

What are the common properties between them ?

- XX^T and $X^T X$ are positive semi-definite matrix
- XX^T and $X^T X$ have same non-zero eigen values
- $\text{rank}(XX^T) = \text{rank}(X^T X) = \text{rank}(X) = r$
- $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_r > 0$

Linear Algebra Fact

The non-zero eigen values of XX^T and $X^T X$ are exactly the same.

$$C = \frac{1}{n} XX^T$$

eigenvectors = $\{w_1, \dots, w_k\}$, such that $\|w_k\|^2 = 1$

eigenvalues = $\{\lambda_1 \geq \dots \geq \lambda_k\}$

$$XX^T = n C$$

eigenvectors = $\{w_1, \dots, w_l\}$, such that $\|w_l\|^2 = 1$

eigenvalues = $\{n \lambda_1 \geq \dots \geq n \lambda_l\}$

$$K = X^T X$$

eigenvectors = $\{\beta_1, \dots, \beta_l\}$, such that $\|\beta_l\|^2 = 1$

eigenvalues = $\{n \lambda_1 \geq \dots \geq n \lambda_l\}$

$$k \beta_k = n \lambda_k \beta_k$$

Is $\beta_k = \alpha_k$?

$$\beta_k^T k \beta_k = \beta_k^T n \lambda_k \beta_k$$

$$\beta_k^T k \beta_k = n \lambda_k \beta_k^T \beta_k$$

$$\beta_k^T k \beta_k = n \lambda_k$$

we can see that there is some scaling of $n \lambda_k$
so, we can set

$$\alpha_k = \frac{\beta_k}{\sqrt{n \lambda_k}}$$

Now

- $k \alpha_k = (n \lambda_k) \alpha_k$
- $\alpha_k^T k \alpha_k = 1$

Possible Algorithm when $d \gg n$

Given dataset

$$\{x_1, x_2, \dots, x_n\} \mid x \in \mathbb{R}^d$$

Step 1 : Compute $K = X^T X$; $K \in \mathbb{R}^{n \times n}$

Step 2 : Compute eigen decomposition of K , eigenvectors = $\{\beta_1, \dots, \beta_k\}$, eigenvalues = $\{n \lambda_1 \geq \dots \geq n \lambda_l\}$

Step 3 : Set $\alpha_k = \frac{\beta_k}{\sqrt{n \lambda_k}} \forall k = 1, \dots, r$

Step 4 : Set $w_k = X \alpha_k \forall k$

Addressing the Non - Linearity Issue

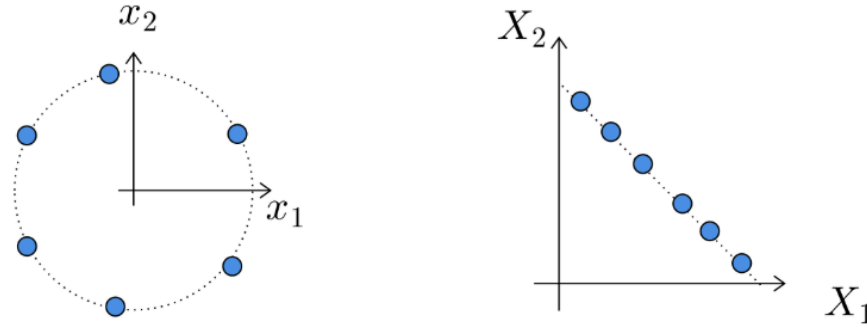


Figure 2: non - linear relationship among data

Consider some data points lying uniformly on a circle with the center at the origin. The relationship between features and datapoints are defined as

$$\begin{aligned} (f_1 - a)^2 + (f_2 - b)^2 &= r^2 \\ f_1^2 + a^2 - 2f_1a + f_2^2 + b^2 - f_2b - r^2 &= 0 \end{aligned} \quad (1)$$

example we will take some datapoint x and transform it to 6D space say $\phi(x)$

$$\begin{bmatrix} f_1 & f_2 \end{bmatrix} \rightarrow \begin{bmatrix} 1 & f_1^2 & f_2^2 & f_1f_2 & f_1 & f_2 \end{bmatrix}$$

now, we will take some arbitrary datapoint in 6D say $u \in \mathbb{R}^6$

$$\begin{bmatrix} a^2 + b^2 - r^2 & 1 & 1 & 0 & -2a & -2b \end{bmatrix}$$

we can observe that each datapoint satisfies

$$\phi(x)^T u = f_1^2 + a^2 - 2f_1a + f_2^2 + b^2 - f_2b - r^2 = 0$$

This implies the datapoints actually lie in a linear subspace of $\mathbb{R}^6$

A circle in 2 dimension can be a line in 6 dimension !

Main Idea

Given datapoints; transform features from low dimension $\mathbb{R}^d$ to $\mathbb{R}^D$ by applying ϕ transformation

$$\begin{aligned} x &\rightarrow \phi(x) \\ \mathbb{R}^d &\rightarrow \mathbb{R}^D \end{aligned}$$

Now, since we increase our dimension to much bigger space(D) from smaller feature space(d), we will use Kernel PCA as we know when $d \gg n$ we look at $\phi(x)^T \phi(x)$ instead of $\phi(x) \phi(x)^T$

Are We All Set ?

$\phi(x) \in \mathbb{R}^d$ may be too hard to compute, for example :

Lets capture a cubic relation with data having 4 features

$$\begin{bmatrix} f_1 & f_2 & f_3 & f_4 \end{bmatrix} \rightarrow \begin{bmatrix} 1 & f_1 & f_2 & f_3 & f_4 & f_1f_2 & \dots & f_1f_2f_3 & \dots & f_1f_3f_4 \end{bmatrix}$$

In general, if we have d features and we map it using pth power transformation. the dimension of the transformed feature space is calculated using the formulae

$${}^{d+p}C_d = \frac{(p + d)!}{d! \times (p + d - d)!} = \frac{(p + d)!}{d! \times p!}$$

Now, Transformed dataset might be linear in the transformed feature space. PCA can be run on this transformed dataset in $\mathbb{R}$.
But explicit transformations can be hard. Kernels help here.

Solved Example

Finding dot product $\phi(x_i)^T \phi(x_j)$ without explicitly finding $\phi(x_i)$ and $\phi(x_j)$

$$x = \begin{bmatrix} a & b \end{bmatrix}$$

$$y = \begin{bmatrix} c & d \end{bmatrix}$$

consider the following polynomial function

$$\begin{aligned} (x^T y + 1)^2 &= \left([a \ b]^T \begin{bmatrix} c \\ d \end{bmatrix} + 1 \right)^2 \\ &= (ac + bd + 1)^2 \\ &= a^2 c^2 + b^2 d^2 + 1 + 2acbd + 2ac + 2bd \\ &= \begin{bmatrix} a^2 & b^2 & 1 & \sqrt{2}ab & \sqrt{2}a & \sqrt{2}b \end{bmatrix} \begin{bmatrix} c^2 \\ d^2 \\ 1 \\ \sqrt{2}cd \\ \sqrt{2}c \\ \sqrt{2}d \end{bmatrix} \\ &= \phi(x)^T \phi(y) \end{aligned}$$

Kernel Function

Any function $K : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ which is a "Valid" map is called a **Kernel function**

Types of Kernels

Polynomial kernel of degree p

$$k(x, y) = (x^T y + 1)^p$$

Gaussian/Radial Basis Kernel

$$k(x, y) = \exp \left(\frac{-\|x - x'\|^2}{2\sigma^2} \right)$$

Interestingly, ϕ in this case maps x to an infinite dimensional space.

But How can we say that give a function $K : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ is a "Valid" kernel ?

Method 1 : Exhibit ϕ map explicitly but that might be hard sometimes !

Method 2 : Mercers Theorem !

$K : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ is a valid kernel if and only if :

- a) K is a symmetric i. e; $K(x, x') = K(x', x)$
- b) for any dataset $\{x_1, \dots, x_n\}$ the matrix $K \in \mathbb{R}^{n \times n}$ where $K_{ij} = K(x_i, x_j)$ is a symmetric and positive semi-definite matrix.

Kernel PCA

Input

Dataset

$$\{x_1, x_2, \dots, x_n\} \mid x \in \mathbb{R}^d$$

Kernel

$$K : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$$

Step 1 : Compute $K \in \mathbb{R}^{n \times n}$ where $K_{ij} = K(x_i, x_j)$

Step 2 : Centering the Kernel

Step 3 : Compute eigen decomposition of K , eigenvectors = $\{\beta_1, \dots, \beta_l\}$, eigenvalues = $\{n \lambda_1 \geq \dots \geq n \lambda_l\}$ and normalise to get

$$\alpha_u = \frac{\beta_u}{\sqrt{n \lambda_u}}$$

Step 4 : $w_k = \phi(x) \alpha_k \rightarrow$ defeats the purpose because for that we need to calculate $\phi(x)$ (So instead of this step to find eigenvector we will directly compute the projection of the datapoint in transformed space)

Modified Step 4 : compute $\phi(x_i)^T w_k = \phi(x_i)^T \left(\sum_{j=1}^n \phi(x_j) \alpha_{kj} \right) = \sum_{j=1}^n \alpha_{kj} \phi(x_i)^T \phi(x_j) = \sum_{j=1}^n \alpha_{kj} K_{ij}$

$$\sum_{j=1}^n \alpha_{kj} K_{ij} \quad \forall k \quad x_i \in \mathbb{R}^d \rightarrow \left[\sum_{j=1}^n \alpha_{1j} K_{ij}, \sum_{j=1}^n \alpha_{2j} K_{ij}, \dots, \sum_{j=1}^n \alpha_{lj} K_{ij} \right]$$

But for a given dataset, the data may be centered, but after applying ϕ map, the kernel might not be centred. we need to get centred kernel to measure the accurate direction.

Kernel Centering

Given $K \in \mathbb{R}^{n \times n}$ where $K_{ij} = k(x_i, x_j) \forall i, j$

Create a new kernel, K^c

$$K_{ij}^c = K_{ij} - \theta_i \mathbb{1}_j^T - \mathbb{1}_i \theta_j^T + \rho \mathbb{1}_i \mathbb{1}_j^T$$

where

$$\theta_i = \frac{1}{n} \sum_{k=1}^n K_{ik} \quad \forall i$$

$$\rho = \frac{1}{n^2} \sum_{i,j} K_{ij}$$

$\mathbb{1}_j$ = Vector which has 1 in the j^{th} position and zero every where. It is a n dimensional vector.

$$\{x_1, \dots, x_n\}$$

$$\{\phi(x_1), \dots, \phi(x_n)\}$$

$$\{\phi(x_1) - \mu, \dots, \phi(x_n) - \mu\}$$

where $\mu = \frac{1}{n} \sum_i \phi(x_i)$

$$(\phi(x_i) - \mu)^T (\phi(x_j) - \mu)$$

$$= \phi(x_i)^T \phi(x_j) - \mu^T \phi(x_i) - \mu^T \phi(x_j) + \mu^T \mu$$

Reference

Arun Raj Kumar, Lecture Slides for Machine Learning Techniques

Karthik Thiagarajan, Machine Learning Techniques Github Notes.

Indian Institute of Technology Madras , CS2007 Course Website

Stanford University, CS229 Machine Learning Notes

Thank You !